

COMPILER CONSTRUCTION LAB

Lab Manual

Bottom-Up Parsing: Operator Precedence Parser

Course Title	Compiler Construction
Lab Number	Lab Session — Operator Precedence Parser
Programme	BS Computer Science (Undergraduate)
Duration	3 Hours
Pre-requisites	Theory of Automata, Top-Down Parsing, Bottom-Up Parsing

1. Lab Objectives

After successfully completing this lab, the student will be able to:

1. Define an operator grammar and recognise when a context-free grammar qualifies as one.
2. Explain the three operator precedence relations and their intuitive meaning in terms of handle boundaries.
3. ~~Compute the LEADING and TRAILING sets of a grammar and use them to derive precedence relations mechanically.~~
4. Construct an operator precedence parsing table for a given grammar.
5. Apply the operator precedence parsing algorithm by hand to determine acceptance or rejection of a string.
6. Implement an operator precedence parser in Language and trace its execution on sample arithmetic inputs.

2. Required Equipment and Software

- Pen and paper for hand computation of LEADING/TRAILING sets and the precedence table.

3. Background Theory

3.1 Motivation

Operator precedence parsing is one of the simplest and historically earliest forms of bottom-up parsing. Its main attraction is that the parser can be built directly from a small two-dimensional table of relations between terminal symbols, without first having to compute the canonical collection of LR items. The technique is widely used in hand-written parsers for arithmetic expressions, formula evaluators, calculator programs and embedded scripting languages, where the grammar is small and dominated by infix operators.

Operator precedence parsers are restricted to a narrow class of grammars called operator grammars, but for that class they are very fast and easy to implement. They are not powerful enough to parse a full programming language, but they are still studied because they illustrate the central idea of bottom-up parsing — the location of handles — in its purest form.

3.2 Operator Grammar

A context-free grammar G is called an operator grammar if it satisfies the following two conditions:

7. No production of G has the empty string (epsilon) on the right-hand side.
8. No production of G has two adjacent non-terminals on the right-hand side.

In an operator grammar every right-hand side is a sequence of grammar symbols in which any two consecutive non-terminals are separated by at least one terminal. Such terminals are called operators, and

they are exactly the symbols whose mutual precedence the parser needs to decide. The classical arithmetic grammar used in the worked example below satisfies both conditions and is therefore a valid operator grammar.

3.3 Operator Precedence Relations

Between any two terminals a and b of an operator grammar at most one of the following three relations holds:

- $a < b$ (a yields precedence to b)
- $a = b$ (a has the same precedence as b)
- $a > b$ (a takes precedence over b)

These relations are not symmetric: it is perfectly possible that $a < b$ but not $b < a$. The intuitive meaning of the relations becomes clear when we use them to detect handles. If at some point during the parse the topmost terminal on the stack is a and the current input symbol is b , then:

- $a < b$ says that the handle has not yet been completely shifted onto the stack; therefore shift b .
- $a = b$ says that a and b together belong to the same handle; therefore shift b .
- $a > b$ says that the handle ends at a ; therefore reduce.

If none of the three relations is defined for the pair (a, b) then a parsing error has been detected. The end-of-input marker $\$$ is treated as a special terminal: it has $\$ < a$ for every a that can begin a sentence and $a > \$$ for every a that can end one.

3.4 Computing Relations from LEADING and TRAILING

Two auxiliary sets are defined for every non-terminal A of an operator grammar.

LEADING(A)

The set of all terminals a such that A can derive a string whose first or second symbol is a . Computed by:

9. If $A \rightarrow a\dots$ then a is in $\text{LEADING}(A)$.
10. If $A \rightarrow B a\dots$ then a is in $\text{LEADING}(A)$, where B is a non-terminal.
11. If $A \rightarrow B\dots$ then $\text{LEADING}(B)$ is contained in $\text{LEADING}(A)$, where B is a non-terminal.

TRAILING(A)

The set of all terminals a such that A can derive a string whose last or second-last symbol is a . Computed by:

12. If $A \rightarrow \dots a$ then a is in $\text{TRAILING}(A)$.
13. If $A \rightarrow \dots a B$ then a is in $\text{TRAILING}(A)$, where B is a non-terminal.
14. If $A \rightarrow \dots B$ then $\text{TRAILING}(B)$ is contained in $\text{TRAILING}(A)$, where B is a non-terminal.

Deriving the Three Relations

Let G be an operator grammar with start symbol S . For every production of G of the form $A \rightarrow X_1 X_2 \dots X_n$ the following rules are applied to all consecutive pairs of grammar symbols:

15. If X_i and X_{i+1} are both terminals, then $X_i =. X_{i+1}$.
16. If X_{i-1} and X_{i+1} are both terminals and X_i is a non-terminal (terminals separated by exactly one non-terminal), then $X_{i-1} =. X_{i+1}$.
17. If X_i is a terminal and X_{i+1} is a non-terminal, then for every b in $\text{LEADING}(X_{i+1})$, $X_i <. b$.
18. If X_i is a non-terminal and X_{i+1} is a terminal, then for every a in $\text{TRAILING}(X_i)$, $a .> X_{i+1}$.

The boundary marker $\$$ is handled by adding $\$ <. a$ for every a in $\text{LEADING}(S)$ and $a .> \$$ for every a in $\text{TRAILING}(S)$.

3.5 The Operator Precedence Parsing Table

All three relations together are stored in a single two-dimensional table indexed by terminal pairs. Conventionally, rows are indexed by the topmost terminal of the stack and columns by the current input symbol. Empty cells represent error entries. Because the relations are between terminals only, the table is small — roughly the square of the number of operators in the grammar — and easy to write down.

3.6 The Operator Precedence Parsing Algorithm

The driver routine maintains a stack initialised with $\$$ at the bottom and reads its input from left to right. At every step it inspects the topmost terminal on the stack (skipping any non-terminals that may sit above it) and the current input symbol, then consults the precedence table:

```

push $ on the stack;
let ip point at the first symbol of input string w$;
repeat forever {
    if (top-of-stack is $ and *ip is $) accept and stop;
    let a be the topmost terminal on the stack;
    let b be *ip;
    if      a <. b   or  a =. b   then { push b; advance ip; }
    else if a .> b   then {
        repeat
            pop the stack
        until the topmost terminal on the stack is related
            by <. to the terminal most recently popped;
    }
    else error();
}

```

Note the elegance of the reduce step: the parser does not need to know which production is being applied. It simply pops everything that lies between two consecutive terminals related by $<.$ (on the left) and $.>$ (on the right). The actual non-terminal that replaces the popped handle is irrelevant for further parsing decisions, so a generic non-terminal such as E may be pushed in its place. This is also why operator precedence parsers do not produce parse trees directly; they produce a sequence of reductions, which is sufficient for evaluation in many practical applications.

3.7 Strengths and Limitations

- Strengths: very simple to implement, no need to build LR item sets, small parsing table, fast at run time, easy to handle ambiguity using explicit precedence and associativity rules.
- Limitations: applicable only to operator grammars; cannot handle epsilon productions or grammars with adjacent non-terminals; provides limited error reporting; the same relation is used for unary and binary operators, which sometimes requires preprocessing of the token stream.

4. Worked Example

4.1 The Grammar

Throughout this example we use the unambiguous arithmetic expression grammar shown below. All productions satisfy the two conditions of an operator grammar (no epsilon and no adjacent non-terminals).

```
E -> E + T   |   T
T -> T * F   |   F
F -> ( E )   |   id
```

4.2 LEADING and TRAILING Sets

Applying the rules from Section 3.4 we obtain:

Non-Terminal	LEADING	TRAILING
E	{ + , * , (, id }	{ + , * ,) , id }
T	{ * , (, id }	{ * ,) , id }
F	{ (, id }	{) , id }

4.3 Construction of the Precedence Table

We now apply the four production-based rules from Section 3.4 once for every production. The contributions of each production are summarised below.

```
From E -> E + T :
  TRAILING(E) .> +   gives   + .> + , * .> + , ) .> + , id .> +
  + <. LEADING(T)   gives   + <. * , + <. ( , + <. id
```

```
From T -> T * F :
  TRAILING(T) .> *   gives   * .> * , ) .> * , id .> *
  * <. LEADING(F)   gives   * <. ( , * <. id
```

```
From F -> ( E ) :
  ( <. LEADING(E)   gives   ( <. + , ( <. * , ( <. ( , ( <. id
  TRAILING(E) .> )   gives   + .> ) , * .> ) , ) .> ) , id .> )
  ( and ) separated by E   gives   ( =. )
```

```
Boundary :
  $ <. LEADING(E)   gives   $ <. + , $ <. * , $ <. ( , $ <. id
  TRAILING(E) .> $   gives   + .> $ , * .> $ , ) .> $ , id .> $
```

4.4 The Operator Precedence Parsing Table

Collecting all the contributions yields the following table. The row label denotes the topmost terminal of the stack and the column label denotes the current input symbol. A blank cell indicates an error entry.

	+	*	(	)	id	\$
+	.>	<.	<.	.>	<.	.>
*	.>	.>	<.	.>	<.	.>
(	<.	<.	<.	=.	<.	
)	.>	.>		.>		.>
id	.>	.>		.>		.>
\$	<.	<.	<.		<.	acc

Note: in this manual the symbols <. and .> are written as a less-than/greater-than sign followed by a dot. In handwritten work and most textbooks the dot is placed underneath the relation symbol, but the meaning is identical.

4.5 Trace for Input $\text{id} + \text{id} * \text{id}$

The trace below records the contents of the stack, the remaining input and the action taken at every step. A generic non-terminal symbol E is pushed onto the stack after every reduction; remember that the parser does not need to know exactly which non-terminal was produced.

Stack	Input	Relation	Action
\$	id+id*id \$	\$ <. id	Shift id
\$ id	+id*id \$	id .> +	Reduce id
\$ E	+id*id \$	\$ <. +	Shift +
\$ E +	id*id \$	+ <. id	Shift id
\$ E + id	*id \$	id .> *	Reduce id
\$ E + E	*id \$	+ <. *	Shift *
\$ E + E *	id \$	* <. id	Shift id
\$ E + E * id	\$	id .> \$	Reduce id
\$ E + E * E	\$	* .> \$	Reduce E*E
\$ E + E	\$	+ .> \$	Reduce E+E
\$ E	\$	\$ = \$	ACCEPT

5. Sample Implementation in C

The following C program implements the operator precedence parser for the grammar of Section 4. The parsing table is encoded as a two-dimensional integer array where the codes 0, 1 and 2 stand for the relations $<$, $=$, and $>$, the code 3 stands for accept, and -1 stands for an error entry. The same driver works for any operator grammar provided the table is regenerated.

5.1 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define LT 0    /* <. */
#define EQ 1    /* =. */
#define GT 2    /* .> */
#define ACC 3
#define ERR -1

/* Terminal indices : + = 0 , * = 1 , ( = 2 , ) = 3 , id = 4 , $ = 5 */
int prec[6][6] = {
    /*      +      *      (      )      id      $      */
    /* + */ { GT, LT, LT, GT, LT, GT },
    /* * */ { GT, GT, LT, GT, LT, GT },
    /* ( */ { LT, LT, LT, EQ, LT, ERR },
    /* ) */ { GT, GT, ERR, GT, ERR, GT },
    /* id */ { GT, GT, ERR, GT, ERR, GT },
    /* $ */ { LT, LT, LT, ERR, LT, ACC }
};

int termIndex(const char *t) {
    if (strcmp(t, "+") == 0) return 0;
    if (strcmp(t, "*") == 0) return 1;
    if (strcmp(t, "(") == 0) return 2;
    if (strcmp(t, ")") == 0) return 3;
    if (strcmp(t, "id") == 0) return 4;
    if (strcmp(t, "$") == 0) return 5;
    return -1;
}

/* Stack of grammar symbols held as strings. */
char *stack[200];
int top = -1;

/* Returns the index of the topmost terminal in the stack,
   skipping any generic non-terminal E that may sit above it. */
int topTerminalIndex(void) {
    for (int i = top; i >= 0; i--) {
        int t = termIndex(stack[i]);
        if (t != -1) return i;
    }
}
```

```

    }
    return -1;
}

void printStack(void) {
    for (int i = 0; i <= top; i++) printf("%s ", stack[i]);
}

int main(void) {
    /* Tokenised input. Append $ as the end marker. */
    char *input[] = { "id", "+", "id", "*", "id", "$" };
    int n = 6, ip = 0;

    stack[++top] = "$";
    printf("%-20s%-20s%-15s\n", "STACK", "INPUT", "RELATION", "ACTION");

    while (1) {
        int ti = topTerminalIndex();
        int a = termIndex(stack[ti]);
        int b = termIndex(input[ip]);
        int rel = prec[a][b];

        printf(""); /* alignment */
        char buf[200] = ""; for (int i = 0; i <= top; i++)
            sprintf(buf + strlen(buf), "%s ", stack[i]);
        printf("%-20s", buf);
        char inp[200] = ""; for (int i = ip; i < n; i++)
            sprintf(inp + strlen(inp), "%s ", input[i]);
        printf("%-20s", inp);

        if (rel == ACC) { printf("%-15s\n", "$ = $", "ACCEPT"); break; }
        if (rel == ERR) { printf("%-15s\n", "-", "ERROR"); exit(1); }

        if (rel == LT || rel == EQ) {
            printf("%-15s %s\n", rel == LT ? "<." : "=.",
                "Shift", input[ip]);
            stack[++top] = input[ip++];
        } else { /* GT : reduce */
            printf("%-15s\n", ">", "Reduce");
            /* Pop until top terminal is related by <. to last popped one */
            char *lastTerm = stack[ti];
            top = ti - 1; /* drop popped layer */
            while (top >= 0) {
                int t = topTerminalIndex();
                if (t < 0) break;
                if (prec[termIndex(stack[t])][termIndex(lastTerm)] == LT)
                    break;
                lastTerm = stack[t];
                top = t - 1;
            }
            stack[++top] = "E"; /* generic non-term */
        }
    }
}

```

```

    }
    return 0;
}

```

5.2 Sample Run

Compiling and executing the program with the input `id + id * id` produces the trace shown below, which matches the hand trace of Section 4.5.

STACK	INPUT	RELATION	ACTION
\$	id + id * id \$	<.	Shift id
\$ id	+ id * id \$	.>	Reduce
\$ E	+ id * id \$	<.	Shift +
\$ E +	id * id \$	<.	Shift id
\$ E + id	* id \$	.>	Reduce
\$ E + E	* id \$	<.	Shift *
\$ E + E *	id \$	<.	Shift id
\$ E + E * id	\$	.>	Reduce
\$ E + E * E	\$	.>	Reduce
\$ E + E	\$	.>	Reduce
\$ E	\$	\$ = \$	ACCEPT

6. Lab Procedure

19. Read and understand the theory section, paying particular attention to the meaning of the three operator precedence relations.
20. On paper, verify that the grammar of Section 4.1 is an operator grammar by checking the two conditions of Section 3.2.
21. Compute the LEADING and TRAILING sets for the same grammar by hand and confirm that your answers match Table 4.2.
22. Apply the four production-based rules to construct the precedence table and confirm that your table matches the one in Section 4.4.
23. Trace the parser by hand on the input $(id + id) * id$ and record the stack and action at every step.
24. Open the development environment and create a new C source file named `op_prec_parser.c`.
25. Type the program from Section 5.1, save it, compile and run it. Verify that the printed trace matches Section 4.5.
26. Modify the program so that it reads the input as a tokenised string supplied at run time, then test it on at least three valid and two invalid expressions.
27. Demonstrate the working program to your lab instructor and answer the post-lab questions.

7. Lab Tasks and Exercises

Task 1 — Verifying the Operator Grammar Property

For each of the following grammars, decide whether it is an operator grammar and justify your answer briefly. (a) $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$. (b) $S \rightarrow aSb \mid ab$. (c) $S \rightarrow (S) \mid \text{epsilon}$. (d) $E \rightarrow E + E \mid E * E \mid id$.

Task 2 — LEADING, TRAILING and the Table

Consider the grammar $S \rightarrow i C t S \mid i C t S e S \mid a$, $C \rightarrow b$. After confirming that the grammar is an operator grammar, compute LEADING and TRAILING for every non-terminal, then construct the operator precedence table. State whether any cell contains more than one relation; if so, the grammar is not operator precedence and the table cannot be used.

Task 3 — Parsing by Hand

Using the precedence table of Section 4.4, draw a complete parsing trace for the input $id * (id + id)$. Report at every step the contents of the stack, the remaining input, the relation that was used and the action taken. State the final outcome.

Task 4 — Generalising the Driver (Bonus)

Modify the C program of Section 5.1 so that the precedence table and the list of terminals are read from an input file rather than being hard coded. Use your generalised driver to parse expressions for the

grammar $E \rightarrow E - T \mid T$, $T \rightarrow T / F \mid F$, $F \rightarrow (E) \mid \text{num}$, where num is a numerical literal. Provide the input file together with two correctly accepted strings and one rejected string.

8. Post-Lab Questions

28. Why does an operator precedence parser refuse to accept grammars with epsilon productions or grammars in which two non-terminals appear next to each other?
29. Explain in your own words why the parser does not need to record exactly which non-terminal is produced after a reduction.
30. In what sense is operator precedence parsing weaker than SLR(1) parsing? Give an example of a grammar that is SLR(1) but not operator precedence.
31. How are unary operators such as a unary minus typically handled in an operator precedence parser?
32. If a parsing table cell contains more than one relation, the grammar is not operator precedence. Suggest two transformations that may sometimes resolve the conflict.

9. Assessment Rubric

Criterion	Marks	Remarks
Correct LEADING and TRAILING sets	20	
Correct operator precedence table	20	
Working implementation of the parser	30	
Successful execution on test cases	15	
Answers to post-lab questions	15	
Total	100	

10. References

33. A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques and Tools, 2nd ed., Pearson, 2007.
34. R. W. Floyd, Syntactic Analysis and Operator Precedence, Journal of the ACM, 10 (3), pp. 316-333, 1963.
35. K. C. Loudon, Compiler Construction: Principles and Practice, PWS Publishing, 1997.
36. D. Grune and C. J. H. Jacobs, Parsing Techniques: A Practical Guide, 2nd ed., Springer, 2008.
37. R. Mak, Writing Compilers and Interpreters: A Modern Software Engineering Approach, Wiley, 2009.